

# CMPE 220 System Software: CPU Design

**Professor:** Ishie Eswar

**Team Members (Group 6):**

Harshitha Vadavalli, 017528576

Parth Patel, 018280535

**Submission Date:** May 9th 2026

# GitHub Repository

GitHub Repository Link: <https://github.com/ParthPatel00/CPU-Design>

Git Clone Link: <https://github.com/ParthPatel00/CPU-Design.git>

# How to Download, Compile, and Run

## Requirements:

- macOS or Linux with GCC installed
- Git
- Make

## Step 1: Download

```
git clone https://github.com/ParthPatel00/CPU-Design.git
```

```
cd CPU-Design/src
```

## Step 2: Build

```
make clean && make
```

Cleans and rebuilds everything. Creates two executables: cpu (the emulator) and asm (the assembler).

## Step 3: Assemble and Run Programs

To run the timer program:

```
./asm ../programs/timer.asm ../programs/timer.bin
```

```
./cpu run ../programs/timer.bin
```

To run the hello world program:

```
./asm ../programs/hello.asm ../programs/hello.bin
```

```
./cpu run ../programs/hello.bin
```

To run the fibonacci program:

```
./asm ../programs/fibonacci.asm ../programs/fibonacci.bin
```

```
./cpu run ../programs/fibonacci.bin
```

**Expected output:**

- hello: Hello, World

- fibonacci: 0 1 1 2 3 5 8 13 21 34

- timer: [TIMER] 3 ... [TIMER] 1

- multiply: 6 (recursive 3 \\* 2)

**Step 4: Verbose Mode (run one at a time)**

```
./cpu verbose ../programs/hello.bin
```

```
./cpu verbose ../programs/fibonacci.bin
```

```
./cpu verbose ../programs/timer.bin
```

```
./cpu verbose ../programs/multiply.bin
```

Shows every Fetch / Decode / Execute cycle in full detail.

## Team Member Contributions

Parth Patel:

Designed overall CPU architecture, instruction set, instruction encoding, and memory map. Implemented emulator modules: `cpu.c` (register file), `alu.c` (arithmetic and flag logic), `control.c` (fetch/decode/execute loop), `bus.c` (memory routing), `memory.c` (64K word array and binary loader), `io.c` (memory-mapped IO devices), and `main.c` (CLI entry point). Created CPU architecture documentation and schematic.

Harshitha Vadavalli:

Implemented two-pass assembler (`assembler.c`, `asm_main.c`). Wrote all four assembly programs: `timer.asm`, `hello.asm`, `fibonacci.asm`, and `multiply.asm`. Created Makefile build targets.